SISTEMAS DISTRIBUIDOS

Procesamiento distribuido

Parte 2 | Clase 22

Cliente asíncrono, servidor despachador y proxy inverso con NGINX.

Integrantes

GONZALEZ MARTINEZ ALBERTO EZEQUIEL
MENDOZA CASTAÑEDA JOSE RICARDO

Fecha

23 de octubre de 2026

Alcance del reporte. Describe la implementación de los tres ejercicios y los resultados de las pruebas realizadas. Las capturas de pantalla solicitadas por el enunciado están pendientes de incorporarse.

1. Objetivo y arquitectura

Construir un sistema que reciba el hash SHA-256 de una palabra y distribuya su búsqueda entre dos nodos trabajadores. Un servidor despachador genera las subtareas, asigna trabajo de forma asíncrona y reúne los resultados. Una interfaz web permite realizar la consulta mediante NGINX.

Espacio de búsqueda

Se consideran palabras de exactamente cinco letras minúsculas del alfabeto a-z. SHA-256 produce una huella de 256 bits, expresada como 64 caracteres hexadecimales. El programa calcula el hash de cada candidato y lo compara con el objetivo; no aplica una operación de descifrado.

El espacio completo contiene **26^5 = 11,881,376 candidatos**. Al fijar la primera letra, cada tarea explora como máximo **26^4 = 456,976 candidatos**. Las 26 tareas cubren el espacio sin superponerse.

Recorrido de una consulta

```
Navegador
    |  HTTP :8090
    v
NGINX (interfaz web y proxy inverso)
    |  /dispatcher
    v
WebServer :8080 -> Aggregator -> WebClient
                          |              |
                          v              v
                  Worker :8081    Worker :8082
                     /task           /task
```

Componente	Responsabilidad
Application	Ejercicio 1: genera tareas, invoca el agregador y muestra resultados.
Aggregator / WebClient	Planificación dinámica y peticiones HTTP asíncronas.
WebServer	Ejercicio 2: recibe el hash por /dispatcher y devuelve el reporte.
ServidorHTTP / FuerzaBrutaHash	Búsqueda de cinco letras; procesamiento de una primera letra por tarea.
NGINX / web/index.html	Ejercicio 3: interfaz y reenvío de solicitudes al despachador.

2. Desarrollo de los ejercicios

Ejercicio 1. Cliente HTTP asíncrono

Application obtiene el hash desde los argumentos; si no se proporciona, usa el de la palabra abaco. Genera las 26 tareas con el formato **hashObjetivo,letra**, desde a hasta z, y las entrega a Aggregator junto con las direcciones de los dos workers.

El agregador inicia una cadena de CompletableFuture por worker. Al terminar una solicitud, el mismo worker recibe la siguiente tarea pendiente. Cada cadena mantiene como máximo una petición en curso. Un estado sincronizado por consulta protege el índice de asignación, los resultados y la condición de parada.

Cuando llega una palabra, se valida su longitud, su primera letra y su hash. Si es válida, se detienen los nuevos envíos. Las tareas ya enviadas terminan y sus respuestas también se incluyen. Los resultados se ordenan por el índice original; el tiempo total incluye esta espera.

Ejercicio 2. Servidor despachador

WebServer atiende **/dispatcher** en el puerto 8080. POST recibe el hash como texto plano; GET admite ?hash=HASH o ?HASH. Cada solicitud crea su propio agregador y sus 26 tareas, evitando compartir el estado de búsqueda entre usuarios.

La respuesta contiene las tareas realizadas, la dirección de cada worker, los resultados, la contraseña encontrada o NULL, el número de tareas y el tiempo total. La cabecera **X-debug-info** incluye la duración en segundos. Un hash inválido devuelve HTTP 400; un fallo de comunicación con un worker devuelve HTTP 502.

Ejercicio 3. Interfaz web y NGINX

La interfaz permite ingresar un hash de 64 caracteres y enviar la búsqueda. Incluye un ejemplo de abaco y muestra la respuesta del despachador usando texto, sin interpretar el contenido como HTML. NGINX sirve la página en el puerto 8090 y reenvía /dispatcher al puerto 8080.

NGINX cumple la función de proxy inverso. La división en subtareas, la asignación a workers y la detección del resultado se realizan en Java. El proxy no sustituye al coordinador de procesamiento distribuido.

Decisiones de implementación

Se conservan Application y WebServer para poder demostrar ambos ejercicios. Se reutiliza el motor de cinco letras de Clase_21. Cada petición del worker utiliza su propio MessageDigest. No se implementan reintentos ni recuperación de tareas ante la caída de un worker.

3. Validación y resultados

Las pruebas se ejecutaron en Windows con Java 26.0.1 y NGINX 1.28.0. Se verificaron dos workers, el despachador y el proxy mediante solicitudes HTTP reales. No se atribuyen estas mediciones a una ejecución en WSL ni a equipos físicamente independientes.

Prueba	Resultado observado
Compilación, JAR y NGINX	Compilación correcta, JAR ejecutable y configuración válida.
Acceso mediante el proxy	Interfaz y /dispatcher disponibles mediante NGINX.
Parada temprana: aaaaa	Palabra encontrada; se ejecutaron 2 de 26 tareas.
Consultas simultáneas	abaco y bbbbb devueltas en consultas independientes.
Búsqueda sin coincidencia	NULL después de completar las 26 tareas.
Validación de entradas	HTTP 400, 404, 405 y 413 en los casos comprobados.
Worker no disponible	HTTP 502, sin informar una búsqueda completa como exitosa.

Respuesta real registrada

Hash objetivo de aaaaa:

```
ed968e840d10d2d313a870bc131a4e2c311d7ad09bdf32b3418147221f51a6e2

Tarea 1 | primera letra: a | worker: localhost:8081 | aaaaa
Tarea 2 | primera letra: b | worker: localhost:8082 | NULL

Contrasena: aaaaa
Tareas realizadas: 2 / 26
Tiempo total: 0.627372 segundos
```

La transcripción resume las líneas de tareas del archivo evidencias/consulta.txt; la contraseña, la cantidad de tareas y el tiempo se conservan del registro. Es una muestra de ejecución, no una captura de pantalla.

Interpretación

La parada temprana evita explorar las letras restantes. El tiempo depende de la carga del sistema, la planificación y el trabajo ya enviado. Dos workers en una misma computadora comparten CPU y memoria; por ello no se puede concluir que el tiempo sea la mitad del de una ejecución secuencial.

4. Ejecución y entrega

Compilar y ejecutar desde WSL

Requisitos: JDK 11 o superior y NGINX de Linux. Antes de iniciar, detener las instancias de Windows para liberar los puertos. Desde WSL:

```
cd "/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/Clase_22"
mkdir -p classes
javac -encoding UTF-8 -d classes source/*.java
```

Ejecutar cada servicio en una terminal distinta, desde Clase_22:

```
java -cp classes ServidorHTTP 8081
java -cp classes ServidorHTTP 8082
java -cp classes WebServer 8080
```

Para demostrar el ejercicio 1, con ambos workers activos:

```
java -cp classes Application
```

La configuración nginx/nginx.conf incluida utiliza rutas relativas al runtime de Windows. Para WSL se debe usar una configuración de Linux con la raíz apuntando a la ruta absoluta de Clase_22/web y el proxy /dispatcher dirigido a http://127.0.0.1:8080. El puerto de la interfaz es 8090.

Archivos que acompañan el reporte

Ubicación	Contenido
source/	Application, Aggregator, WebClient, WebServer y motor de búsqueda.
web/ y nginx/	Interfaz web, configuración y runtime local de NGINX.
README.md	Instrucciones de ejecución, pruebas y capturas.
tests/integracion.py	Pruebas de integración con dos workers y NGINX.
evidencias/	Respuesta registrada y descripción de las verificaciones.

Evidencia gráfica pendiente

Para completar los requisitos gráficos de la entrega se deben incorporar: **(1)** una captura de /dispatcher con su resultado y tiempo total, y **(2)** una captura del navegador con una consulta junto a la terminal del despachador. Estas capturas no están incluidas en este documento.

Conclusión

La implementación divide la búsqueda en 26 partes, las asigna dinámicamente a dos workers y coordina la parada cuando se encuentra una coincidencia. El despachador expone el cálculo mediante HTTP y NGINX permite acceder desde una interfaz web. Las pruebas verificaron el flujo completo y la separación del estado entre consultas.

Referencia: M. en C. Ukranio Coronilla, Procesamiento Distribuido, Parte 2, enunciado de la práctica proporcionado. Base del proyecto: Clase_19 y Clase_21 del repositorio del curso.